Source Certified Region Types and Observational Summaries

Mettapedia Working Notes

31 October 2026

Abstract

Five Lean modules extend algebra inference along three boundaries. A proof-producing elaborator connects a pure natural fragment of actual PeTTa atoms to the existing operational semantics and additive certificates. A separate word worker combines those certificates with current-program and overflow checks, preserving profile-indexed results and declared interleaving boundaries. Tropical expressions receive ordered, bag or set permissions from their derived algebra. Finally, finite-label mode has an exact future-addition quotient, a compact representation and a complete input-dependent separating test family. These results connect optimization to refinement, effect and quotient typing without claiming a complete MeTTa type inference algorithm, native C verification or automatic performance optimality.

1 Region evidence has several different roles

A useful optimization judgment must identify more than a convenient source shape. The checked pipeline separates the following evidence:

Evidence	What it licenses
Source elaboration	Interpreting captured variables and supported primitives in the declared source profile
Algebra certificate	Replacing an admitted state fold by its input-only summary
Live refinement	Representing current values and arithmetic by machine words
Observation profile	Preserving final values, logical counts, or prefix states as required
Dependency identity	Reusing evidence only for the current program it describes
Contextual quotient	Forgetting exactly the distinctions invisible to admitted future observations

These are related to familiar type-theoretic techniques, but are not identical to ordinary expression-type inference. The elaborator synthesizes a typed IR and its derivation; a certificate is indexed by the program it proves; a word plan contains data justified by a live arithmetic refinement; the reply type depends on the requested observation profile. Unknown admission means ordinary execution, not a new source-language error.

There is relevant precedent. Datafun's type-and-syntax-directed transformations extend semi-naive evaluation to higher-order programs and prove preservation through a logical relation [1]. Gradual Type Theory studies when type-based optimizations preserve contextual equivalence [2]. The results here are specialized preservation and erasure laws, not a full graduality theorem or an implementation of either cited language.

2 A source elaborator with operational meaning

`PeTTa.Bridges.AggregationSource` consumes the existing `Atom` carrier. A role environment identifies captured input and accumulator variables. Its accepted syntax consists of nonnegative integer literals, those variables, addition, multiplication by a literal natural on the left, and the indicator expression

```
if (input == label) then 1 else 0.
```

The elaborator returns an IR expression together with a structural elaboration derivation. The semantics theorem is independent of this recognizer: induction on the derivation uses the existing PeTTa arithmetic and conditional execution laws to establish `PureReturns`. Under the captured-natural-value environment, the original source atom returns exactly one corresponding natural answer and leaves the PeTTa space state unchanged.

The source-program classifier combines those derivations with the earlier additive inference algorithm. A successful certificate gives, coordinate by coordinate,

$$\mathrm{sourceStep}(s, x)_i = s_i + w_i(x).$$

It is constructed from source atoms and the computed coefficient test, not from a caller-supplied assertion of the entire source equivalence. Positive controls include a counter expressed through an input-equality indicator and its reassociated spelling. Negative controls decline an effectful expression, duplicate-producing choice, a negative literal, an unknown capture and an indicator depending on the accumulator rather than the input. Multiplication by zero does not hide an effectful argument: source elaboration must first qualify that argument before pure algebraic normalization applies.

The captured environment is an explicit entry precondition; this package does not infer all variable roles or validate every runtime binding automatically. The arithmetic profile is the existing PeTTa operational model, where those native primitives evaluate their arguments. General user-equation unfolding, whole tuple/fold-body extraction, other dialects and source budget boundaries remain additional bridges. A declined elaboration says nothing about whether ordinary MeTTa evaluation of that atom is meaningful.

3 Static algebra and live word realization

`InferredWordFold` keeps a static `Certificateprogram`. At a live request it checks that the current IR program still equals that program and that every initial coordinate plus its accumulated contribution fits in `UInt64`. An accepted plan stores words and executes actual word addition. The proof reuses the existing checked word-addition theorem, rather than defining the worker to call the natural reference.

The representation check is separate from the algebra check. A source may be additive but currently exceed the word range. Conversely, small current values do not make an order-dependent source additive. Since the extracted natural contributions are nonnegative, the final bound also bounds every prefix accumulator. The theorem does not claim that planning, conversion or evaluating the weights is free.

Requests have dependent result types:

- a value request returns the final register vector;
- a counted request returns the logical number of fold steps and that vector;
- a prefix request returns every source prefix state.

The word worker handles the first two and declines prefix requests. Its ordinary fallback preserves the complete result type. Tests cover overflow without wraparound, stale cached bodies, preserved logical step count and profile-dependent admission. The cache comparison is structural program equality; a cheap native revision stamp needs a separate correspondence proof.

The worker is a concrete instance of the existing polynomial cursor's `PartialWorker`. Its local refinement supplies a provider morphism, so adaptive clients and arbitrary declared interleavings preserve replies, current state, residual continuation and completion. Environment actions may change registers or the current program between requests; admission is checked again against live state.

A run request already denotes a completed atomic IR block. This proof does not authorize merging source operations across yield points, callbacks or budget stops. The counted profile measures IR fold steps, not PeTTa evaluator instructions. Native allocation, failure and linearization also remain implementation obligations.

`AggregationWordBridge.word_coordinate_returns` closes a concrete source-to-worker square: every admitted one-step word result is an answer of the elaborated source coordinate in the existing PeTTa semantics, with one answer occurrence and unchanged space state. A checked example computes its certificate from source atoms and proves that initial state 5, input 7 and a matching indicator return 6. Batch refinement separately preserves the extracted IR fold. This does not yet certify an entire native MeTTa call ABI.

4 Inferring which algebra permits reordering

`TropicalRegionInference` compiles state, constants, natural shifts and minimum expressions to the existing tropical affine monoid. A compiled update acts on $\mathbb{N} \cup \{\infty\}$ as

$$x \longmapsto \min(x + a, b).$$

The source-expression evaluation and compiled action are proved equal. Ordered products summarize arbitrary finite runs and preserve concatenation.

Two semantic classification tests are exact for their named functional forms:

$$\begin{aligned} \forall x,\ \min(x + a, b) = \min(x, b) &\iff a = 0 \ \vee \ b = 0, \\ \forall x,\ \min(x + a, b) = x + a &\iff b = \infty. \end{aligned}$$

The zero-cap case admits a shifted expression whose state contribution is semantically irrelevant. It would be missed by checking only whether the printed shift was zero.

Caps commute and are idempotent; shifts commute. The batch classifier selects the cap family, the shift family or ordered affine composition. Theorems license permutations of each admitted commutative batch and erasure of an adjacent repeated cap. General ordered summarization remains available.

The algebra parameter matters. Capping at 3 and then shifting by 5 maps zero to 5; shifting first and then capping maps it to 3. Each operation belongs to a commutative family, but their union does not. A generic "commutative" flag cannot safely join unrelated families. Likewise, cap final values can be idempotent while the source's changed/not-changed events still distinguish orders. The permissions concern final values, not all possible observations.

5 The exact finite-label mode quotient

Fix $K \geq 1$ labels, ordered $0, \ldots, K-1$, and natural counts h_i. The mode is absent for the zero bag; otherwise it is the smallest maximal-count label. Define the differences and occupancy:

$$D_i(h) = h_i - h_0 \in \mathbb{Z}, \qquad E(h) = [h \neq 0].$$

The anchor difference is known to be zero. The compact encoding stores only $K-1$ differences and the occupancy bit, with an exact expansion/readout law. Differences add under bag addition and occupancy combines by Boolean OR.

`FiniteModeInformation.sameFuture_iff_summary` proves

$$(\forall \delta \in \mathbb{N}^K,\ \mathrm{mode}(h+\delta) = \mathrm{mode}(g+\delta)) \quad \Longleftrightarrow \quad (D(h), E(h)) = (D(g), E(g)).$$

Thus every summary that answers all admitted future additions must distinguish these same classes. The decoder computes the mode from the signed differences, and its result is linked to the earlier sorted source fold on finite labels. No physical bit-packing or globally shortest-program claim is needed.

Necessity is constructive. If a pair of count differences disagrees, add enough votes to raise that pair above all other labels in both stores. Choose the added counts so that one store ties at the smaller label while the other strictly selects the larger. The resulting vote vector is a separating future context. A zero addition distinguishes a difference in occupancy.

The implementation enumerates zero and two such boosts for each label relative to the anchor: $2K+1$ candidates, including redundant anchor cases. The proved `separator` returns a distinguishing addition when one exists, and returns none exactly when the summaries are equal. This finite family depends on the two input stores; it is not a fixed test suite proving arbitrary program equivalence. Its size bounds the candidate count, not CPU time.

The fixed universe is essential. Counts $(1,1)$ and $(2,2)$ have the same future mode behavior on two labels. Introduce a third label with two votes, and $(1,1,2)$ selects that new label while $(2,2,2)$ selects the smallest old one. Total count and deletion observations also need information deliberately erased by this quotient. Representation changes must carry their universe and observer contract, not only the compressed numbers.

6 Connection to the S work and remaining boundaries

For S1's representation work, the finite-label result supplies a precise condition for replacing counts by an observational summary and for detecting lost information. It does not permit erasing general relation multiplicity, occurrence identity or holder lifetimes. For S6's cursor algebra, the inferred word worker is a concrete typed realization under the existing adaptive-client and interleaving laws. These interfaces also identify what a future S3 decision-diagram/transducer backend must preserve, but that backend is not implemented or certified by this package.

The resulting type-inference connection is a composition of source judgments, algebraic certificates, observation grades, live refinements and quotient indices. The types supply executable permission tests and checked transports. They do not choose the fastest physical plan without cost evidence, force all programs into a Horn fragment, or replace the surrounding language semantics.

The next work is broader source/profile extraction, typed symbolic-transducer admission, native identity/lifetime and cost refinement, and qualification against equivalent source spellings and adverse observer contexts. The current source judgment and native-state simulation must remain independent of any future C implementation they are used to qualify.

References

[1] Michael Arntzenius and Neel Krishnaswami, "Seminaïve Evaluation for a Higher-Order Functional Language," *Proceedings of the ACM on Programming Languages* 4, POPL, article 22, 2020. doi:10.1145/3371090.

[2] Max S. New, Daniel R. Licata and Amal Ahmed, "Gradual Type Theory," extended version, 2018; POPL 2019. arXiv:1811.02440.